Aula 20 – Estruturas de Dados Heterogêneas (struct)

Objetivo da aula: Compreender o conceito de estruturas heterogêneas, aprender a declarar uma struct, acessar seus campos e utilizar typedef para organizar dados de diferentes tipos em um programa C.

1. Relembrando os conteúdos anteriores

Até agora, aprendemos a trabalhar com variáveis, vetores, matrizes, strings e funções. Imagine que precisamos cadastrar um aluno com nome, idade e nota.

```
char nome[50];
int idade;
float nota;
```

Temos três variáveis diferentes, mas todas representam informações de uma mesma pessoa. Existe uma maneira de reunir essas informações em um único conjunto?

Sim! Utilizando struct.

2. O que são estruturas de dados?

Na programação, frequentemente precisamos armazenar e organizar várias informações. ***Uma estrutura de dados é uma maneira de organizar essas informações para que possam ser armazenadas, acessadas e utilizadas pelo programa.***

Nesta aula, vamos comparar duas formas de organização: homogênea e heterogênea.

3. Estruturas de dados homogêneas

São estruturas que armazenam elementos do mesmo tipo de dado. Por exemplo, um vetor que armazena cinco notas:

```
float notas[5] = {7.5, 8.0, 6.5, 9.0, 10.0};
```

Todos os elementos são do tipo float. Outro exemplo é uma matriz:

```
int numeros[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

4. Estruturas de dados heterogêneas

São estruturas que permitem agrupar informações de diferentes tipos de dados. Imagine que precisamos cadastrar um aluno com as seguintes informações:

- Nome: Maria (char[50])
- Idade: 20 (int)
- Nota: 8.5 (float)

Essas informações são diferentes, mas pertencem ao mesmo aluno. Em Linguagem C, podemos agrupá-las utilizando uma struct.

5. Comparando as duas estruturas

Característica	Homogênea	Heterogênea
Tipos de dados	Mesmo tipo	Podem ser diferentes
Exemplos em C	Vetores e matrizes	`struct`
Organização	Por posições	Por campos
Acesso	`notas[0]`	`aluno.nota`
Aplicação	Armazenar várias notas	Cadastrar informações de um aluno

6. O que é uma struct?

Uma struct é uma estrutura que permite agrupar informações de diferentes tipos em um único conjunto. Por exemplo, um aluno pode ter nome (char), idade (int) e nota (float).



7. Como declarar uma struct?

```
struct Aluno {
    char nome[50];
    int idade;
    float nota;
};
```

Explicação linha por linha:

- **struct Aluno**: cria um modelo de estrutura chamado Aluno.
- **char nome[50]:** reserva espaço para o nome.
- **int idade**: armazena a idade.
- **float nota**: armazena a nota.
- **};**: encerra a definição da estrutura.

Para criar uma variável desse tipo, escrevemos:

```
struct Aluno aluno1;
```

A variável aluno1 possui os três campos definidos na estrutura.

8. Como acessar os campos?

Utilizamos o operador ponto (.).

```
aluno1.idade = 20;
aluno1.nota = 8.5;
```

Isso significa que estamos armazenando valores nos campos idade e nota da variável aluno1.

Para apresentar os valores:

```
printf("Idade: %d\n", aluno1.idade);
printf("Nota: %.1f\n", aluno1.nota);
```

9. Primeiro programa completo – Cadastro de aluno

```
#include <stdio.h>

struct Aluno {
    char nome[50];
    int idade;
```

```
    float nota;
};

int main(void) {
    struct Aluno aluno1;

    printf("Digite o nome: ");
    scanf(" %49[^\n]", aluno1.nome);

    printf("Digite a idade: ");
    scanf("%d", &aluno1.idade);

    printf("Digite a nota: ");
    scanf("%f", &aluno1.nota);

    printf("\n--- DADOS DO ALUNO ---\n");
    printf("Nome: %s\n", aluno1.nome);
    printf("Idade: %d\n", aluno1.idade);
    printf("Nota: %.1f\n", aluno1.nota);

    return 0;
}
```

Exemplo de execução:

Digite o nome: Maria Silva
Digite a idade: 20
Digite a nota: 8.5

--- DADOS DO ALUNO ---
Nome: Maria Silva
Idade: 20
Nota: 8.5

aluno1 representa um aluno e que cada campo armazena uma informação específica.

10. O que é typedef?

O typedef permite criar um nome alternativo para um tipo de dado. No exemplo anterior, escrevemos:

```
struct Aluno aluno1;
```

Podemos simplificar essa declaração utilizando typedef:

```
typedef struct {
    char nome[50];
    int idade;
    float nota;
} Aluno;
```

Agora podemos declarar:

```
Aluno aluno1;
```

Importante: typedef não cria automaticamente um aluno. Ele apenas permite utilizar o nome Aluno como um tipo de dado.

10.1 Programa completo utilizando typedef

```
#include <stdio.h>

typedef struct {
    char nome[50];
    int idade;
    float nota;
} Aluno;

int main(void) {
    Aluno aluno1;

    printf("Nome: ");
    scanf(" %49[^\n]", aluno1.nome);

  printf("Idade: ");
    scanf("%d", &aluno1.idade);

    printf("Nota: ");
    scanf("%f", &aluno1.nota);

    printf("\nAluno: %s\n", aluno1.nome);
    printf("Idade: %d\n", aluno1.idade);
    printf("Nota: %.1f\n", aluno1.nota);

    return 0;
}
```

O funcionamento é o mesmo. A diferença está na forma de declarar a variável.

11. Diferença entre struct com e sem typedef em Linguagem C

Em Linguagem C, podemos declarar estruturas de duas maneiras: utilizando apenas struct ou combinando typedef com struct.

A principal diferença está na forma como declaramos as variáveis. O funcionamento da estrutura é o mesmo.

11.1 struct sem typedef

Primeiro, definimos a estrutura:

```
struct Aluno {
    char nome[50];
    int idade;
    float nota;
};
```

Para criar uma variável, precisamos escrever a palavra struct seguida do nome da estrutura:

```
struct Aluno aluno1;
```

Exemplo completo:

```
#include <stdio.h>

struct Aluno {
    char nome[50];
    int idade;
    float nota;
};

int main(void) {
    struct Aluno aluno1 = {"Maria", 20, 8.5};

    printf("Nome: %s\n", aluno1.nome);
    printf("Idade: %d\n", aluno1.idade);
    printf("Nota: %.1f\n", aluno1.nota);

    return 0;
}
```

11.2 struct com typedef

O typedef permite criar um nome alternativo para o tipo da estrutura.

```
typedef struct {
    char nome[50];
    int idade;
    float nota;
} Aluno;
```

Agora podemos declarar uma variável diretamente com o nome Aluno:

```
Aluno aluno1;
```

12. Resumo:

Conceito	O que aprendemos
Estrutura homogênea	Armazena elementos do mesmo tipo.
Estrutura heterogênea	Permite agrupar campos de tipos diferentes.
`struct`	Define uma estrutura com campos.
Operador `.`	Permite acessar os campos.
`typedef`	Cria um nome alternativo para um tipo.

"Organizar bem os dados é tão importante quanto saber processá-los. Com struct, conseguimos representar informações do mundo real de maneira mais clara dentro dos nossos programas. "